

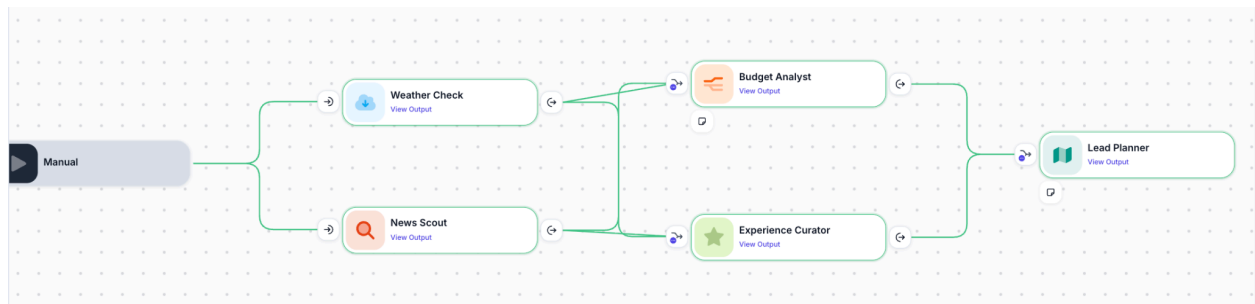
Joseph Sifuentes
Agentic AI
Spring 2026
03/25/26

Final Project: Project Milestone 2 - Multi-agent workflow

Affordable Travel Guide

Link to my travel agent:

<https://app.vellum.ai/public/workflow-deployments/66d55b72-0ba8-4dcc-90d9-fb28ba50d825?releaseTag=LATEST>



Project Overview: The Multi-Agent Budget Travel Orchestrator is a specialized AI system designed to provide hyper-realistic, low-cost travel itineraries for students, educators, and low-income travelers. By moving away from a single-prompt model and utilizing a coordinated team of five distinct AI agents, the system ensures that every recommendation is cross-referenced for safety, weather compatibility, and financial feasibility. The core philosophy of the orchestrator is "Budget-First," meaning the system defaults to the most economical options, such as hostels, public transit, and free cultural activities, while strictly avoiding luxury recommendations unless explicitly requested.

Agent Roles and Specifications: The system is divided into three functional phases. In the first phase, the **Meteorologist** and **News Scout** act as the environmental gatekeepers. The Meteorologist tracks live weather patterns for both the origin and destination, providing seasonal risk assessments to ensure travelers aren't sent into dangerous conditions like hurricane or monsoon seasons. Simultaneously, the News Scout monitors real-time safety advisories, local strikes, and major events that could cause price

surges or travel disruptions. If a destination is deemed unsafe or physically inaccessible, these agents have the authority to veto the location before further planning occurs.

In the second phase, the **Budget Analyst** and **Experience Curator** handle the tactical modeling of the trip. The Budget Analyst serves as the financial lead, calculating a strict "Daily Spend Limit" based on the user's total budget and group size. This agent is programmed to prioritize student and teacher discounts, applying logic for specific perks like ISIC card rates to every cost estimate. Working within the financial boundaries set by the Analyst, the Experience Curator identifies "Hidden Gems" that are either free or cost under \$15. This agent focuses on public parks, museum "free days," and local street food to ensure the traveler receives a high-quality cultural experience without financial strain.

Communication Architecture The orchestrator follows a Directed Acyclic Graph (DAG) workflow to manage data dependencies effectively. The process begins with a "Fan-Out" protocol where the Meteorologist and News Scout trigger in parallel to gather environmental context. Once this data is secured, it "feeds in" to the Budget Analyst, who adjusts the financial model based on the reported external conditions. For example, if the News Scout reports a transit strike, the Analyst reallocates funds to cover more expensive alternative transportation. Finally, the **Lead Planner** acts as the executive synthesizer, merging the structured data from all sub-agents into a cohesive, hourly, day-by-day narrative.

Failure Handling and Quality Assurance. To ensure the integrity of the budget-travel promise, the system employs several safety checks. A "Value Check" gatekeeper prevents the system from suggesting any activity or lodging that exceeds the pre-defined daily cap; if a high-cost item is suggested, the Lead Planner rejects the input and requests a budget alternative. Additionally, the system uses "AWAIT_ATTRIBUTES" logic, which prevents the generation of a final report until all necessary data points—such as confirmed weather and verified costs—are received. This multi-layered approach eliminates "hallucinated" costs and ensures that the final itinerary is both safe and strictly affordable.

Multi-agent Budget Travel Orchestrator



CODE:

POST v1/execute-workflow

```
# pip install vellum-ai
```

```
import os
```

```
from vellum.client import Vellum
```

```
import vellum.types as types
```

```
# create your API key here: https://app.vellum.ai/organization?tab=workspaces&item=environments
```

```
client = Vellum(
```

```
    api_key=os.environ["VELLUM_API_KEY"]
```

```
)
```

```
result = client.execute_workflow(
```

```
    workflow_deployment_name="affordable-travel-guide",
```

```
    release_tag="LATEST",
```

```
    inputs=[
```

```
        types.WorkflowRequestStringInputRequest(
```

```
            name="budget",
```

```
            type="STRING",
```

```
            value="<example-string-value>",
```

```
        ),
```

```

        types.WorkflowRequestStringInputRequest(
            name="location",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="trip_length",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="travel_group",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="transportation",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="preferences",
            type="STRING",
            value="<example-string-value>",
        ),
    ],
)

```

```

if result.data.state == "REJECTED":
    raise Exception(result.data.error.message)

```

```

print(result.data.outputs)

```

NEXT:

[POST v1/execute-workflow-stream](#)

```

# pip install vellum-ai

```

```

import os

```

```

from vellum.client import Vellum

```

```

import vellum.types as types

```

```

# create your API key here: https://app.vellum.ai/organization?tab=workspaces&item=environments

```

```

client = Vellum(

```

```

    api_key=os.environ["VELLUM_API_KEY"]

```

```

)

```

```

event_stream = client.execute_workflow_stream(

```

```

    workflow_deployment_name="affordable-travel-guide",

```

```

    release_tag="LATEST",

```

```

    inputs=[

```

```

        types.WorkflowRequestStringInputRequest(

```

```

        name="budget",
        type="STRING",
        value="<example-string-value>",
    ),
    types.WorkflowRequestStringInputRequest(
        name="location",
        type="STRING",
        value="<example-string-value>",
    ),
    types.WorkflowRequestStringInputRequest(
        name="trip_length",
        type="STRING",
        value="<example-string-value>",
    ),
    types.WorkflowRequestStringInputRequest(
        name="travel_group",
        type="STRING",
        value="<example-string-value>",
    ),
    types.WorkflowRequestStringInputRequest(
        name="transportation",
        type="STRING",
        value="<example-string-value>",
    ),
    types.WorkflowRequestStringInputRequest(
        name="preferences",
        type="STRING",
        value="<example-string-value>",
    ),
],
)

```

```

outputs = {}
for event in event_stream:
    # listen to only WORKFLOW events
    if (event.type != "WORKFLOW"):
        continue

    # Use a REJECTED event to handle errors
    if event.data.state == "REJECTED":
        raise Exception(event.data.error.message)

    # You can use Workflow STREAMING events to collect outputs and output deltas as they are emitted
    if (event.data.state == "STREAMING" and event.data.output):
        # If you have multiple Final Output nodes, you can filter by name here.
        # if event.data.output.name not in [travel_plan]:
        #     continue

    if event.data.output.delta is not None:
        if outputs.get(event.data.output.name) is None:
            outputs[event.data.output.name] = event.data.output.delta
        else:
            outputs[event.data.output.name] += event.data.output.delta
    elif event.data.output.value is not None:

```

```

        outputs[event.data.output.name] = event.data.output.value

# Alternatively, all outputs will be available in the fulfilled event
if (event.data.state == "FULFILLED" and event.data.outputs):
    outputs = event.data.outputs

print(outputs)

```

NEXT

POST v1/execute-workflow-async

```

# pip install vellum-ai
import os
import time
from vellum.client import Vellum
import vellum.types as types

# create your API key here: https://app.vellum.ai/organization?tab=workspaces&item=environments
client = Vellum(
    api_key=os.environ["VELLUM_API_KEY"]
)

# Submit the workflow execution request
initiated_response = client.execute_workflow_async(
    workflow_deployment_name="affordable-travel-guide",
    release_tag="LATEST",
    inputs=[
        types.WorkflowRequestStringInputRequest(
            name="budget",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="location",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="trip_length",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="travel_group",
            type="STRING",
            value="<example-string-value>",
        ),
        types.WorkflowRequestStringInputRequest(
            name="transportation",
            type="STRING",

```

```

        value="<example-string-value>",
    ),
    types.WorkflowRequestStringInputRequest(
        name="preferences",
        type="STRING",
        value="<example-string-value>",
    ),
],
)

execution_id = initiated_response.execution_id

# Poll for the result
while True:
    result = client.workflows.workflow_execution_status(execution_id=execution_id)

    if result.status == "REJECTED":
        raise Exception(result.error.message)
    elif result.status == "FULFILLED":
        print(result.outputs)
        break
    elif result.status == "INITIATED":
        # Workflow is still running, wait before polling again
        time.sleep(1)
    else:
        # Workflow is in an intermediate state, continue polling
        time.sleep(1)

```

NEXT

POST /v1/submit-workflow-execution-actuals

```

# pip install vellum-ai
import os
from vellum.client import Vellum
from vellum.types import (
    WorkflowExecutionActualStringRequest,
)

# create your API key here: https://app.vellum.ai/organization?tab=workspaces&item=environments
client = Vellum(
    api_key=os.environ["VELLUM_API_KEY"]
)

client.submit_workflow_execution_actuals(
    execution_id="<id-visible-from-execution-details-page>",
    actuals=[
        WorkflowExecutionActualStringRequest(
            output_key="travel_plan",
            quality=1.0, # 1.0 is good, 0.0 is bad
            output_type="STRING",
            desired_output_value="A value representing what you wish the output had been",

```

```
    metadata={},
),
WorkflowExecutionActualStringRequest(
    output_key="weather",
    quality=1.0, # 1.0 is good, 0.0 is bad
    output_type="STRING",
    desired_output_value="A value representing what you wish the output had been",
    metadata={},
),
WorkflowExecutionActualStringRequest(
    output_key="news",
    quality=1.0, # 1.0 is good, 0.0 is bad
    output_type="STRING",
    desired_output_value="A value representing what you wish the output had been",
    metadata={},
),
WorkflowExecutionActualStringRequest(
    output_key="budget_analysis",
    quality=1.0, # 1.0 is good, 0.0 is bad
    output_type="STRING",
    desired_output_value="A value representing what you wish the output had been",
    metadata={},
),
WorkflowExecutionActualStringRequest(
    output_key="experiences",
    quality=1.0, # 1.0 is good, 0.0 is bad
    output_type="STRING",
    desired_output_value="A value representing what you wish the output had been",
    metadata={},
),
]
)
```